

Switch Scheduling

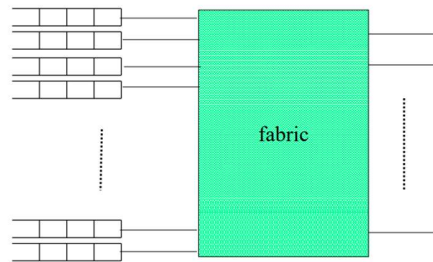
Switch: Simplified Model

יש לנו מתג פשוט עם N כניסות ויציאות. את הפקטות מחלקים לחלקים באותו הגודל הנקראים cells. נניח שלכל הפורטים יש אותה המהירות וננרמל אותה ל1, כלומר כל מתג יכול לפנות ולהעביר תא אחד בכל יחידת זמן.

שיטות תזמון scheduling

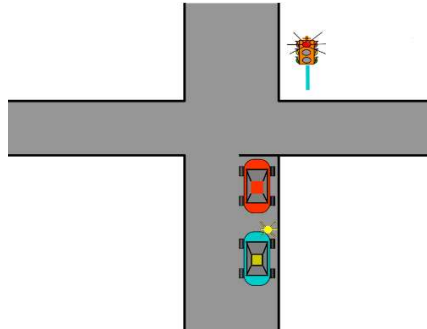
1. Input Queuing

החבילות מגיעות ועוברות דרך באפרים (ביציאה אין באפרים). ה-fabric מעביר חבילה אחת והיתר ימתינו.



זה דומה לצומת

כאן מותר לפנות ימינה באדום. נניח שמגיעה המכונית האדומה והיא רוצה לנסוע ישר. מאחוריה מגיעה מכונית שרוצה לפנות ימינה אבל היא לא יכולה לפנות כי המכונית האדומה חוסמת אותה. זאת תופעה שנקראת head of line blocking.



Head of Line (HOL) Blocking

נניח שלחבילות שממתינות בראש התור יש יעד אקראי (בזרות מס' אחיד בין 1 ל- n). השאלה היא כמה יעדים שונים נבחרו? לשם כך נסתכל על התוחלת של היעדים השונים:

$$E[\# \text{ distinct outputs}] = \sum_{i=1}^n Pr[\text{some input has output } i] \stackrel{\text{משלים}}{=} n \left(1 - \left(\frac{n-1}{n} \right)^n \right) \rightarrow n \left(1 - \frac{1}{e} \right) \approx 0.63n$$

יהיה לנו אינדיקטור שאומר האם נבחר היעד או לא נבחר. התוחלת תהיה שווה לסכום ההסתברויות של האינדיקטורים האלו.

אם n קלטים בוחרים באקראי פלט מס' הפלטים השונים בתוחלת הוא פחות מ- $\frac{2}{3}n$.

זה מקרה מדומיין כי היעדים אינם אקראיים, יש יעדים שיותר פופולריים.

אבל אנחנו רואים כאן שהיעילות של המתג לא תהיה מאוד גבוהה.

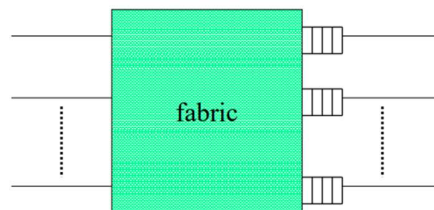
בעיה: הכי פשוט למימוש אבל בעל ביצועים גרועים.

2. Output queuing

אנחנו לא ניתן לאף חבילה לחכות בכניסה, כל החבילות שמגיעות מיד יפנו למוצא ולכן נשים באפרים במוצא.

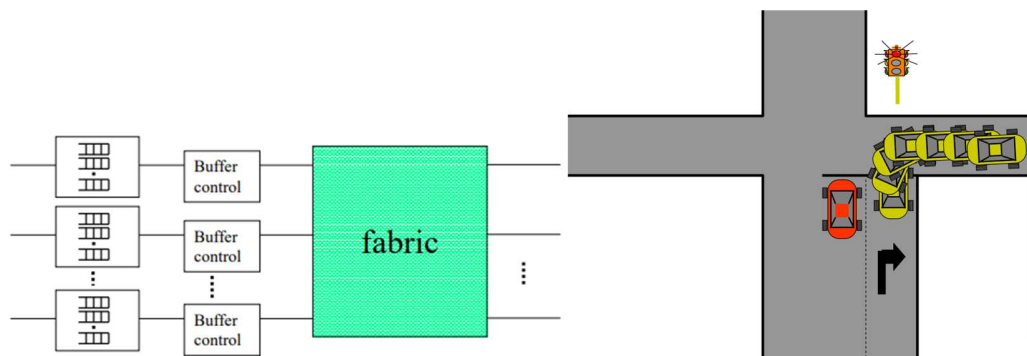
בעיות:

- ביטלנו את הבעיה של ה-HOL, אף חבילה לא תמתין בגלל חבילה שלא אמורה להפריע לה. אבל יש עדיין המתנות במוצא כי יכול להיות שיש כמה חבילות. אבל עם זה אין מה לעשות. כיוון שכל החבילות מחכות במוצא אפשר להסתכל על כולן ולהחליט את מי בדאי לקדם (נקרא משטרי תורים (queuing disciplines), למשל אם יש חבילה בעלת אילוצי real time וכו'.
- Output queuing זה דבר די דמיוני כי אנחנו צריכים שה-fabric יפנה את כל החבילות, לא משנה באיזה תבנית, אל המוצא clock אחד. המהירות של ה-fabric צריכה להיות כמו סכום המהירויות של הקלטים (פי N). כלומר זה אידיאלי מבחינת ביצועים אבל קשה למימוש.



3. Virtual Output Queuing

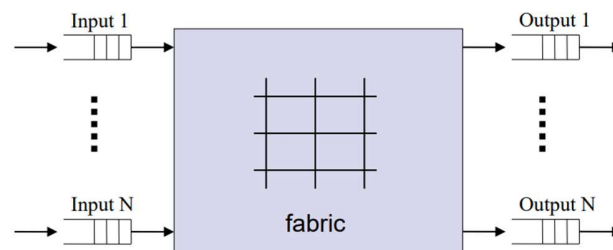
באיור מטה ניתן לראות שאם המכונת האדומה מחכה לאור ירוק היא לא תפריע למי שרוצה לפנות ימינה כי יש להם נתיב נפרד. כלומר הרעיון virtual output queuing הוא לעשות את הבאפרים בכניסות אבל בכל כניסה למיין את החבילות לפי היעדים.



כלומר מס' הבאפרים גדל מ- n ל- n^2 .
זה הרעיון הכי נפוץ היום.

4. Combined Input Output Queuing (CIOQ) Switch

כעת נראה רעיון יותר מתוחכם שיעבוד טוב לא רק ביעדים אקראיים אלא גם במקרה הגרוע ביותר. יש לנו באפרים בכניסות כמו בinput queuing ובאפרים ביציאות כמו בoutput queuing. כאן תהיה לנו האצה קטנה יותר, ה-fabric יעבוד בקצב גבוה פי s כאשר $1 < s < N$. אנחנו נקבל כל דבר שoutput queuing יכול לעשות. נראה איך בהינתן תסריט שיכול לעשות מתג של output queuing אנחנו יכולים לבצע את אותו תסריט ע"י מתג של CIOQ ששם speedup יהיה רק 2!



Mimic OQ with CIOQ switch

Scheduling Framework

השאלה היא מה ההאצה הכי קטנה שאפשר בשביל לעשות כל דבר שoutput queuing switch יכול לעשות?
האצה של 2 מוגדרת כ:

נסתכל על יחידת זמן אחת (קצב הקווים בכניסה וביציאה) ונפרק אותה ל-4 תת צעדים:

1. חבילות מגיעות לכניסה
 2. fabric מעביר פרמוטציה חלקית
 3. חבילות מגיעות ליציאה
 4. fabric מעביר פרמוטציה חלקית
- כלומר בכל פעם הוא פתר שני פרמוטציות.

The stable marriage problem | אלגוריתם נישואים יציבים

זה אלגוריתם מתורת המשחקים.

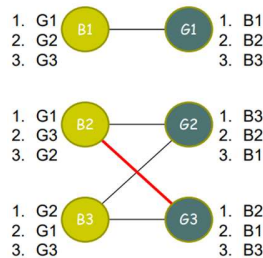
יש לנו n בנים $\{b \in B\}$ ו- n בנות $\{g \in G\}$.

כל בת מדרגת את כל n הבנים $r_g(b)$, היא תתן n נקודות למי שהיא הכי מחבבת, $n-1$ למי שהיא פחות מחבבת וכו'. באופן דומה כל בן מדרג את כל הבנות $r_b(g)$. נניח שהאפשרות הכי פחות מועדפת היא להשאר לבד.
עכשיו נסתכל על זיווג, כלומר n זוגות. נאמר שהזיווג הזה הוא יציב אם מתקיים התנאי הבא:
אם אין לנו זוג חוסם, כלומר זוג שלא זיווגו אותו אחד לשני אבל כל אחד מהם היה מעדיף את האחר על פני מי שהוא מזווג אליו.
לכן הזוג הזה רוצה להרוס את מבנה החברה ולהרוס את היציבות.

דוגמה

יש לנו את הבנים בצהוב ואת הבנות באפור. הדירוגים שלהם מסומנים ליד.

הזוג הראשון תקין. בזוג השני הבן מעדיף את $G1$ אבל היא משודכת למישהו אחר ו- $G3$ מעדיפה את $B2$.
כלומר $B2$ ו- $G3$ הם זוג חוסם (מסומן בקו אדום).



•Matching:
B1-G1, B2-G2, B3-G3.
Stable?

•No! B2 - G3 blocking

•Matching:
B1-G1, B2-G3, B3-G2
Stable?

•Why?

Stable marriage algorithm | האלגוריתם

הגדרה: זיווג חלקי

M – זיווג חלקי

B_M, G_M – בנים ובנות מזווגים ב- M

$$M(b) = g \text{ if } (b, g) \in M \quad M(g) = b \text{ if } (b, g) \in M$$

בונים את הזיווג היציב באופן איטרטיבי ומתקיים.

נבחר בת שאינה מזווגת $g \in G$, נסתכל על הרשימה של הבת הזאת ועוברים על הבנים ברשימה הזאת אחד אחד. אנחנו נעצרים כאשר:

1. מגיעים לבן הראשון שאינו מזווג $b \notin B_M$ ואז מזווגים ביניהם

$$\rightarrow M := M \cup \{(b, g)\}$$

2. מגיעים לבן הראשון שהוא כן מזווג אך הוא מעדיף את הבת הנוכחית על פני בת זוגו $b \in B_M$ and $r_b(g) > r_b(M(b))$.
לכן מזווגים ביניהם וזורקים את בת הזוג הישנה.

$$\rightarrow M := (M \cup \{(b, g)\}) - \{(b, M(b))\}$$

עוצרים כאשר כולם מזווגים: $|M| = n$

למה בוודאות אפשר למצוא מיישור ברשימה שמקיים את 2 התנאים האלה?
 g היא אינה מזווגת ולכן בהכרח יש b שאינו מזווג לכן אנחנו חייבים להגיע למיישור שאינו מזווג.

למה האיטרציות מתכנסות? הרי אנחנו עושים כל הזמן החלפות
 קל לראות שכן – נסתכל על הבנים. כל בן בהתחלה שאינו מזווג, יש לו את הרשימה שלו ומתחת לכל הרשימה יש את הבת הכי פחות מועדפת. אז כל הבנים מתחילים עם פוינטר על השורה האחרונה ברשימה שלהם לא מזווגת. כאשר יש איטרציה כזאת אז יש איזה שהוא בן ש: או שהיה לא מזווג ומזדווג לבת, או שמוצאים לו בת זוג טובה יותר. בכל מקרה עבור הבן שהזדווג באיטרציה הזאת, הפוינטר שלו זז למעלה – או מלא-מזווג למזווג, או ממזווג למזווג בדירוג גבוה יותר. בכל מקרה לאחד הבנים המחוג מתקדם בכל איטרציה.
 בגלל שיש n בנים ו-n בנות לאחר n^2 איטרציות נגיע לזיווג יציב.

למה 1

בכל איטרציה הזיווג החלקי M שחישבנו עד עכשיו יציב.

בגלל שאם $r_g(b) > r_g(M(g))$ אז $b \in B_M$ וגם $r_b(g) < r_b(M(b))$

למה 2

לאחר n^2 איטרציות נגיע לזיווג יציב.

בהינתן זיווג חלקי M, נגדיר את הפוטנציאל:

$$\phi(M) = \sum_{b \in B_M} r_b(M(b))$$

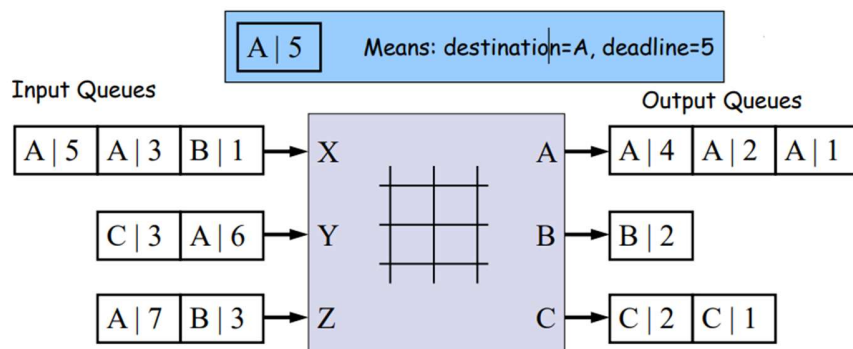
$\phi(M)$ גדל לפחות ביחידה אחת כל איטרציה ולא יכול לחרוג מ- n^2 .

Input & output priority list

בתזמון מתג יש לנו פורטים של כניסה (בנים) ופורטים של יציאה (בנות). כל אחד מהם ידרג את הפורטים. נשתמש באלגוריתם *stable marriage* וכך נעביר את הפקטות מהכניסה ליציאה.

- *output ports* מדרגים את הs: התא הכי דחוף, שצריך להעביר הכי מהר, יהיה בעל הדירוג הגבוה יותר.
 - הדרך שבה הs מדרגים את הs יותר מסובכת:
- נניח שיש לנו cell בפורט הכניסה X שהולך לפורט המוצא A ודדליין t. הדירוג יהיה 1 ועוד המספר של החבילות בפורט היציאה A עם דדליין לפני t.

דוגמה



מספיק speedup של 2

טענה

האלגוריתם יכול לדמות כל מתג OQ תחת כל $pattern$ הגעות.

הגדרה: עבור יחידת זמן t , $L_t(c)$ של תא c עם כניסה X ויציאה A הוא:

מס' התאים ב- A עם $deadline$ יותר קרוב פחות מס' התאים לפני c ב- X :

$$L_t(c) = \#(\text{cells in } A \text{ with closer deadline}) - \#(\text{cells preceding } c \text{ in } X)$$

באופן אינטואיטיבי: כמה דחוף התא c .

אם תא c מגיע בזמן t אז: $L_t(c) = 0$.

זה פקטור 2 כי בזוג יש לנו בן ובת שצריך לזווג ולכן צריך 2 פעימות כדי להבטיח את זה.

למה

לכל $t \geq 0$ מתקיים $L_{t+1}(c) \geq L_t(c)$.

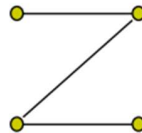
זה לא הגיוני להריץ אלגוריתם עם סיבוכיות n^2 פעמיים. לכן משתמשים באלגוריתמים מקורבים אחרים:

Practical Algorithms

נשים לב שבכל פעם שה- $fabric$ פועל הוא מממש זיווג בין הכניסות למוצאים. המטרה: למצוא את הזיווג הכי גדול – זיווג מקסימום.

יש הבדל בין:

- **זיווג מקסימום** – הזיווג הכי גדול שיש.
 - **זיווג מקסימלי** – זיווג שאי אפשר להגדיל אותו. זאת קבוצה של קשתות ואי אפשר להוסיף לה עוד קשת, כל קשת נוספת תיגע באחת מהקשתות הקיימות.
- בדוגמה מטה זיווג המקסימום הוא 2 (העליונה והתחתונה) והזיווג המקסימלי הוא 1 (רק הקשת האלכסונית).



זיווג מקסימום הוא גם זיווג מקסימלי אבל לא להפך.

אפשר להראות שכל זיווג מקסימלי הוא מכיל לפחות מחצית ממספר הקשתות בזיווג מקסימום:

$$maximal \geq \frac{1}{2} maximum$$

1. Practical Algorithm: PIM (Parallel Iterative Matching)

אלגוריתם פשוט לתזמון של מתגים שמנסה למצוא זיווג מקסימלי.

○ הקלטים שולחים בקשות למוצאים

לכל פורט כניסה ופורט יציאה יש כרטיס רשת והם מדברים אחד עם השני כדי להבין מה לבקש מה- $fabric$ לשלוח.

○ המוצא בוחר אחד מהקלטים באקראי ושולח לו GRANT

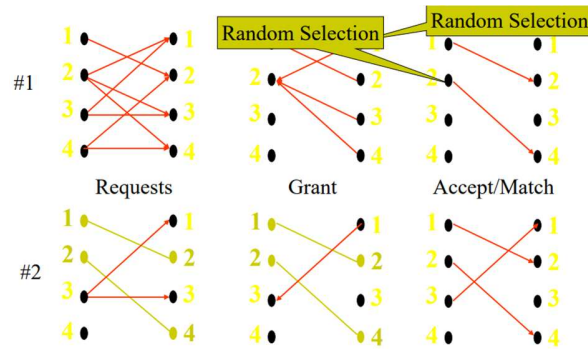
בעת יש מוצאים שקיבלו הרבה הודעות, קצת הודעות או לא קיבלו בכלל. המוצא בוחר באקראי את אחד המקורות ששלחו לו הודעות ואומר "אני מוכן ללכת איתך".

○ כל קלט בוחר באקראי את אחד המוצאים ששלח לו GRANT ושולח לו ACCEPT

הקלט שלח הרבה הודעות להרבה מוצאים, לכן הוא בוחר את אחד מהם. את אלה שבחרו אחד את השני אנחנו מכניסים לזיווג.

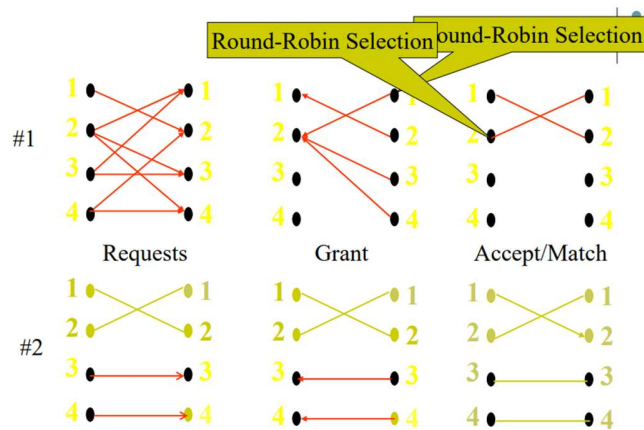
עושים עוד איטרציה על הכניסות והיציאות שאינן מזווגות. אפשר לראות שאחרי $O(\log(n))$ נגיע לזיווג המקסימלי.

דוגמה



2. iSLIP

האלגוריתם הפופולרי בראוטרים בשנים האחרונות. הוא דומה ל*PIM* חוץ מזה שהבחירה האקראית מומרת בבחירה דטרמינסטית. כל פורט מחזיק מחוג על לוח מעגלי, הוא מבצע *round robin*, עובר על כל הפורטים ובוחר את הראשון הפנוי. מקודם ה-*GRANT* וה-*ACCEPT* היו רנדומליים ועכשיו אנחנו בוחרים לפי המצב הנוכחי.



יתרונות:

אם ה-*load* נמוך (לא מגיעות הרבה חבילות) הבחירות נראות אקראיות. כאשר ה-*load* גבוה (בכל כניסה יהיו חבילות להרבה פורטים), המחוג של *iSLIP* מתקדם כל פעם אחד וזה נראה כמו *round robin* דטרמיניסטי. העדיפות הכי נמוכה זה מי שקיבל בפעם האחרונה. כדי לממש את זה משתמשים ב-*N priority encoders*.

Traffic analysis

כעת נדבר על תזמון ביציאה מהמתג.

A model for traffic streams

f – זרם של ביטים שמגיע.

$R^f(t)$ – קצב הזרימה של f היא המהירות בביטים לשניה בזמן t .

B – מס' הביטים הכולל.

מס' הביטים הכולל בזרם בזמן $[x, y]$ מוגדר ע"י:

$$B^f(x, y) = \int_x^y R^f(t) dt$$

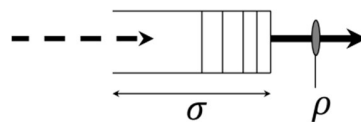
Network element: a buffer

לבאפר יש 2 תכונות:

- σ – הגודל שלו.

- ρ – קצב הריקון שלו.

נתעניין בשאלה איזה זרמים יכולים להכנס מכאן כך שהבאפר הזה לא יגלוש?



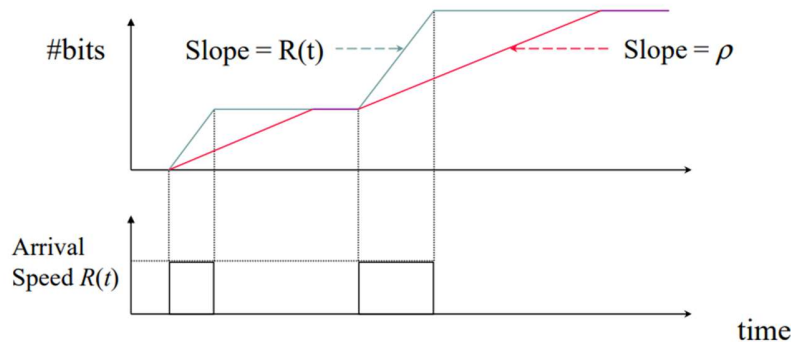
Example: Two packets arrive at a buffer

נתבונן שוב בתהליך הגעה ותהליך עזיבה. הגרף התחתון מתאר את קצב ההגעה – זה דומה לקצב הגעה של חבילות.

הגרף העליון מתאר את מס' הביטים:

- הגרף הירוק – מס' הביטים של תהליך ההגעה.

- הגרף האדום – מס' הביטים של תהליך העזיבה.

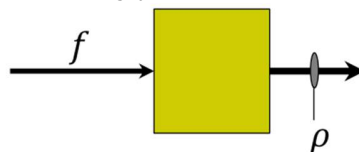


הגדרה: backlog (פיגור)

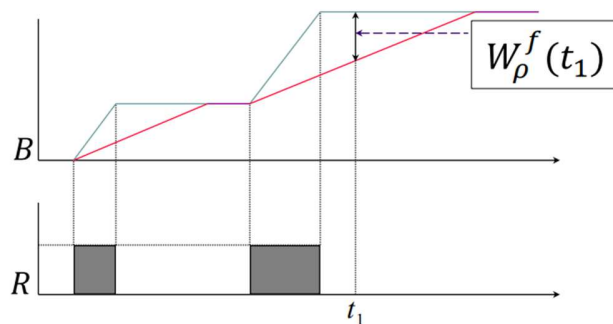
אלו ההזמנות שהצטברו, כלומר זה מס' הביטים שעדיין אגורים לנו בבאפר. מסמנים את ה-backlog באות W .

העיכוב בזמן t : נסתכל על כל הזמנים לפני t ונשאל כמה ביטים הגיעו מ- s עד t ונוריד מזה כמה ביטים יכלו לצאת $\rho(t-s)$. אנחנו צריכים את המקסימום כי הביטוי מתעתע, זה לא נכון שבכל חלון זמן מ- s עד t יצאו $\rho(t-s)$, בגלל שיכול להיות שהביטים בכלל לא הגיעו.

$$W_p^f(t) = \max_{s \leq t} \{B^f(s, t) - \rho(t-s)\}$$



ניתן לראות מבחינה גרפית שה-*backlog* הוא בדיוק ההפרש המקסימלי באיור:



זרמים שמוגבלים ע"י (σ, ρ) constrained streams

הגדרה

- בהינתן $\rho \geq 0$ ו- $\sigma \geq 0$ נכתוב $R \sim (\rho, \sigma)$ אם לכל הזמנים $y \geq x$ מתקיים: $B(x, y) \leq \sigma + \rho(y - x)$.
זה בדיוק מתאר את הזרמים שיכולים להקלט בבאפר בגודל σ בקצב ניקוז ρ בלי לאבד אף ביט.
אינטואיציה: זה מתאר חוסר דריסה.
- אם זרם שמוגבל ע"י (σ, ρ) מגיע בחבילות בגודל L ביטים על קו במהירות C , אפשר לחשב את גודל הבאפר ע"י:

$$\sigma \geq \frac{L}{C}(C - \rho) = L \left(1 - \frac{\rho}{C}\right)$$